SENG3320 Assignment 2 Report

Alex Rubin 3486124 and Jackson Bray 3479565

Fuzzer Design

Architecture of the Fuzzer

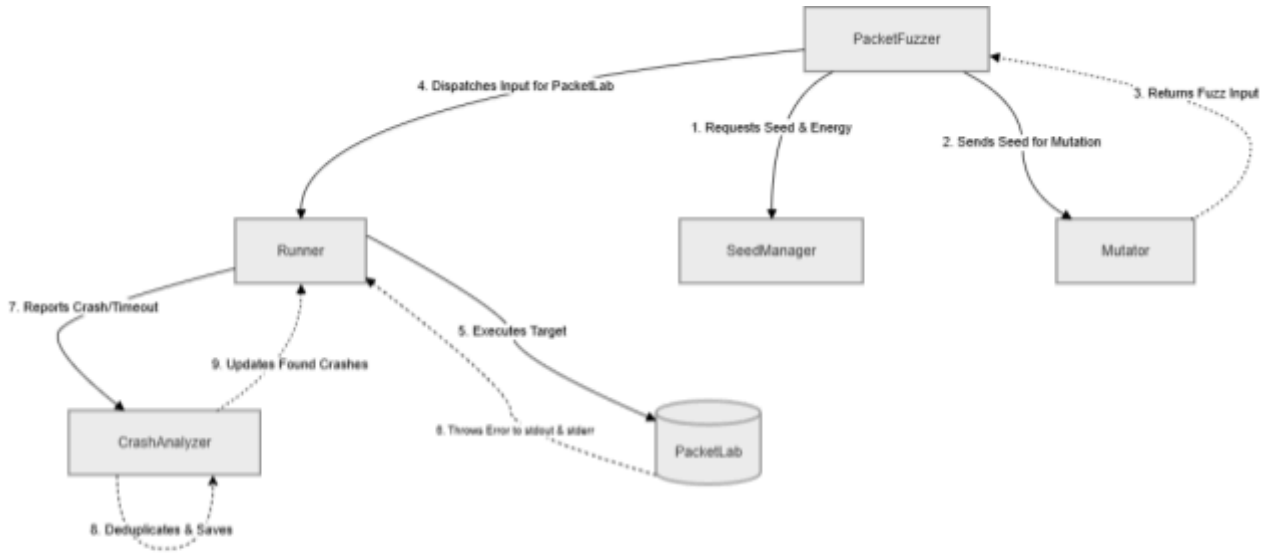

Figure 1: A high level diagram describing the modularity and flow of the PacketFuzzer program

The architecture of the fuzzer is designed as mutational and seed-based. The fuzzer is implements a Modular Orchestration pattern. Rather than a single script which handles the entire process, the program is split into five distinct modules (described in Figure 1) with each handling a specific subset of tasks in the fuzzing lifecycle including: The Orchestrator (PacketFuzzer), Seed Generation, Seed Mutation, Input Runner, and Crash Analysis modules.

Seed Selection

Seeds are managed by the ***SeedManager*** module. Before additional directives are added, a set of betwee 1 and 4 packet seeds are added to the file. Then the module utilizes a pseudo-random selection strategy to choose between a pool of viable directive inputs via getNextSeed(). The pool of viable inputs contains two categories:

- **Valid Structured Directives:** Where the directives are exactly as the program expects and intended
  - **Example:** `"AVG|10.0.0.1"`
- **Syntactically correct but malicious directives:** Where the directives are exactly as the program expects syntactically, but differs in how the program intends for the directive to be utilized.
  - **Example:** `"SPIN|-1"`

By starting with structured seeds as opposed to random noise, the fuzzer ensures it bypasses the initial parsing stage of the PacketLab class to reach deeper execution paths within the program. Whilst on a technicality, a purely noise-driven fuzzer would achieve 100% coverage of exceptions and errors given enough time, the utilization of seeds trades some accuracy for a lot of unnecessary computation and time. Furthermore, by retaining the two classes of seed directives, the efficiency of the fuzzer is drastically improved, and in some cases reached additional crashes that were otherwise unreachable.

How is Energy Assigned

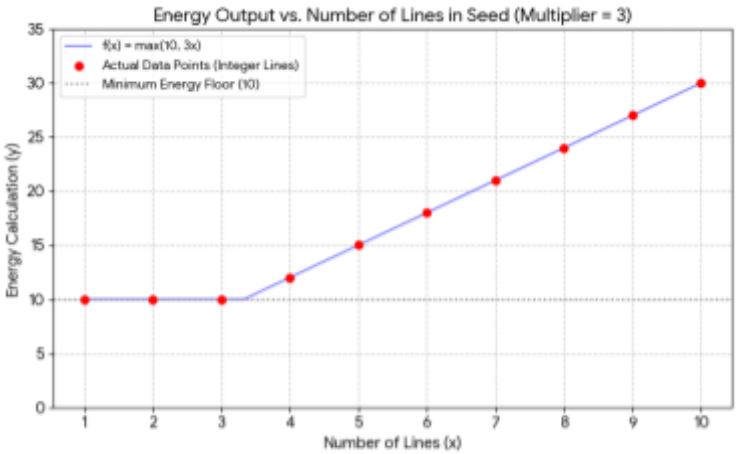

Figure 2: Line graph describing the Energy Assignment function.

In addition to the seed generation logic, the SeedManager module also handles the logic for the assignment of energy to the seeds. Energy within the fuzzing program represents the number of mutations attempted upon a single seed before the next seed is chosen. In our particular solution, our team utilizes a length-based heuristic in which the amount of energy assigned to a seed is directly related to the number of lines of a seed with a minimum energy of 10. Our rationale comes from the relationship between the length of a seed and its complexity. Longer seeds typically contain more fields and delimiters, presenting more opportunities for potential vulnerabilities compared to smaller seeds. Hence, a line-based approach to the assignment of energy was both the simplest and the most intuitive choice. Figure 2 describes the relationship between a seed's length and its energy as well as the 10 energy minimum that is exceeded once a seed's line length passes ~3.5.

How modules interact during a fuzzing iteration

A single iteration of the fuzzer follows a strictly coordinated sequence with PacketFuzzer as the central orchestrator between the sub-modules and managing the flow of data. The lifecycle of an iteration can be broken down into; (a) The Selection Phase, (b) The Transformation phase, (c) The Execution Phase, and (d) The Analysis & Feedback Phase.

Selection Phase

The iteration begins with the PacketFuzzer (Orchestrator) which queries the SeedManager for the base input. During this phase, the SeedManager executes its **getNextSeed()** logic which combines a range of 1-4 packet seeds, with 1-4 directive OR malicious directive seeds. Once a seed is selected, it's energy is calculated within the SeedManager's **calculateEnergy()** function, described in Figure 2. This value effectively determines the number of mutations and execution steps are executed before sampling a new seed. Once calculated, the energy is returned back to the Orchestrator to begin the Transformation Phase.

Transformation Phase

Once the Orchestrator holds the value for the seed and its energy, it passes the data to the Mutator module. The mutator applies non-deterministic transformation to the input seed by randomly selecting a mutation strategy from the following:

- **Character Flipping:** Flipping a random character in the seed.
- **Value Arithmetic:** Replacing any sequence of digits with a potentially malicious payload described in INT_PARTITIONS
- **Line duplication/deletion:** Where the line is duplicated or deleted entirely.
- **Delimiter Tampering:** Where additional delimiters are added or existing ones are removed.
- **Command Name Mutation:** Where the command is converted to lowercase.
- **Field Deletion:** Where a random field from within the command is deleted.
- **Payload Swapping:** Depending on the particular command, such as PKT, the payload (data) is changed to a subset of potentially malicious payloads.
- **Random line insertion:** An additional line is added below a randomly selected line from a pre-defined pool of additional line seeds.
- **Swap Line:** Two random lines are swapped within the seed.

These specific changes made to a seed are designed to bypass any parsing logic and reach deeper regions within the PacketLab program. The mutator is invoked repeatedly by the Orchestrator until all the seed's energy is used up. At which point, a new seed is chosen.

Execution Phase

The malformed input generated in the Transformation phase are handed to the Runner module which is responsible for executing the commands within the given program. It acts as the bridge between the fuzzer and the target program.

The Runner handles the execution of the program, and collects any raw exit codes and stdout/stderr outputs generated by the target. The Runner packages the exit codes and stack traces and passes them to the Crash Analyser Module.

If it is determined that the program has hung (500ms has passed without exiting), then the program is reran using jcmd to collect the thread dump of the program. This is used to isolate the stack trace of the hung program to produce an accurate crash signature for deduplication.

Analysis and Feedback Phase

In the final phase of the fuzzer, the CrashAnalyzer module receives the outputs and stack traces (or thread dump if the program hung) from the Runner module and performs filtering upon the exit codes and outputs. If the program exited normally or is a duplicate of an existing error, the result is discarded. If the crash is unique, It is saved as a unique file which contains the error code, the stack trace, and the line which caused the crash. Additionally, the error code and some extracted details from the stack trace are concatenated and entered into a HashSet to track for duplicate errors on future runs.

Input Generation Strategy

The input generation strategy of a fuzzer is the most critical module in the program, as it dictates how many exceptions and hangs are discovered throughout the process, and how long the process itself takes. The PacketLab fuzzer utilizes a pool of initialized seeds to “kickstart” the fuzzing process without running into parsing errors.

The program first adds 1-4 PKT commands to the program to ensure the program is multi-line and can reach additional logic. Mutating away from an initially valid seed increases the likelihood of bypassing the parsing logic and triggering an exception significantly compared to a purely pseudo-random generation strategy.

Our program then utilized two types of directive seeds: Valid Structured Directives; which reflect the intended structure and use of a directive, and Malicious Directive seeds; which reflect the intended structure but deviate from the intended use of a directive. The combination of these two seed types ensure a deeper penetration into the program’s logic. The seed inputs included:

```
private static final List<String> packetSeeds = Arrays.asList(
        "PKT|10.0.0.1|10.0.0.2|DATA|4|ABCD",
        "PKT|10.0.0.2|10.0.0.3|CTRL|3|XYZ",
        "PKT|10.0.0.1|10.0.0.4|DATA|5|HELLO",
        "PKT|192.168.0.10|10.0.0.5|DATA|2|OK"
);

private static final List<String> directiveSeeds = Arrays.asList(
        "COUNT|DATA",
        "AVG|10.0.0.1",
        "SLICE|NETWORK|1|4",
        "MERGE|-|foo|bar",
        "REPEAT|AB|3",
        "LOOKUP|10.0.0.1",
        "WINDOW|5",
        "HEX|ff",
        "TOP|2"
);

private static final List<String> maliciousLines = Arrays.asList( // Contain additional
directives that are more likely to cause crashes
        "AVG|99.99.99.99",
        "SPIN|-1",
        "LOOKUP|1.2.3.4",
        "MERGE||a|b",
        "WINDOW|10",
        "REPEAT|A|-1",
        "TOP|9999",
        "HI"
);
```

Once a list of seeds are pseudo-randomly selected from the pool described above, a mutation strategy was also pseudo-randomly selected from it's own pool:

- **Character Flipping:** Flipping a random character in the seed.
    - **What part of the input is fuzzed:** Character flipping can freely flip any character in the entire seed.
- **Value Arithmetic:** Replacing any sequence of digits with a potentially malicious payload described in INT_PARTITIONS
    - **What part of the input is fuzzed:** Value Arithmetic can fuzz any numeric parameter within a directive.
- **Line duplication/deletion:** Where the line is duplicated or deleted entirely.
    - **What part of the input is fuzzed:** Line duplication and deletion influence the entire seed
- **Delimiter Tampering:** Where additional delimiters are added or existing ones are removed.
    - **What part of the input is fuzzed:** Delimiter Tampering mutates the pipe symbols that separate the packet fields, numeric parameters, payload text, and command names.
- **Command Name Mutation:** Where the command is converted to lowercase.
    - **What part of the input is fuzzed:** Exclusively the command names are targeted within command name mutation.
- **Field Deletion:** Where a random field from within the command is deleted.
    - **What part of the input is fuzzed:** Field Deletion targets any field in between two pipe delimiters.
- **Payload Swapping:** Depending on the particular command, such as PKT, the payload (data) is changed to a subset of potentially malicious payloads.
    - **What part of the input is fuzzed:** Payload swapping impacts the data sent by specific commands such as the 6th slot in the PKT command, or the 2nd/3rd slot in the MERGE command.
- **Random line insertion:** An additional line is added below a randomly selected line from a pre-defined pool of additional line seeds.
    - **What part of the input is fuzzed:** Random line insertion targets entire lines, to exploit potential vulnerabilities with paired directives.
- **Swap Line:** Two random lines are swapped within the seed.
    - **What part of the input is fuzzed:** Random line insertion targets entire lines, to exploit potential vulnerabilities with order of directives and other related commands.

By specifically crafting mutation strategies for the format and use-cases of PacketLab rather than relying solely on pseudo-random char flips, the mutation became significantly more likely to spot exceptions.This is due to the fuzzer's ability to maintain the original structure of a directive/seed, and instead focus on altering particular features like its payload or IP address.

Crash Detection and De-duplication

Crash detection is handled by the Runner module which executes PacketLab and classifies any outputs into crashes or hangs. If PacketLab returns a non-zero exit code, it is classified as a crash. For these cases, the runner captures the stdout and stderr streams and forwards it to the analyser to produce a signature and check for uniqueness.
Alternatively, if the program times out and does not produce an exit code for 500ms, the runner classifies the run as a hang and reruns the program using jcmd, and collects the thread dump. If the jcmd produces a thread dump successfully, the program identifies the specific line related to PacketLab and uses it as the signature for the hang. This way the program can still measure if a hang is unique or not, without the stack traces from stdout and stderr in a traditional thrown exception.
Once a unique signature is formed, it is checked for uniqueness within a hash set and if returned true, entered into the same hash set. This solution ensures there are no duplicate results within the crash logs of the program for both standard exceptions, and program hangs.

Fuzzing Results

The program was executed 20,000 times in order to obtain a high coverage. It was able to discover 28 unique crashes. For each crash the program creates a folder under “/crashes”. These folders contain a crash report which provides the reason for the crash, the stack trace of the program at the time of the crash, as well as the inputs that caused the crash. The inputs that caused the crash are also included within the crash folder.

Improving the Fuzzer

As of the current iteration, the mutator applies mutation strategies in a random fashion. This is functional, but wastes energy (computationally wasteful) as it can explore areas that it has already explored before. This could be improved by removing previously explored seeds from the pool, and replacing them with the mutated variants. This would allow the fuzzer to explore new branches or patterns that would otherwise take longer to achieve “randomly”. Additionally, if some metric could be calculated to determine the “riskiness” of a seed, then more computation could be driven towards those avenues, further increasing the computational efficiency of the program.

Symbolic Execution and Mutation Testing

Symbolic Execution

KLEE Execution

KLEE was run using docker. The steps on the KLEE website were followed for how to set up and use KLEE with docker. These instructions can be found at https://klee-se.org/docker/. After creating and entering the docker container, the docs at https://klee-se.org/releases/docs/v3.2/tutorials/testing-function/ were used to learn how to use klee to test a function.

First a file called `program.c` was created. It contained the triangle program that was to be tested, as well as additional code to properly set up symbolic execution with klee. It was created with the command below:

```
echo '/* The Triangle program, which determines if three inputs specify an
equilateral
triangle, an isosceles triangle, an ordinary triangle, or a non-triangle. */
#include <stdio.h>
#include "klee/klee.h"

void triangle(int a, int b, int c);

int main() {
    int a, b, c;
    klee_make_symbolic(&a, sizeof(a), "a");
    klee_make_symbolic(&b, sizeof(b), "b");
    klee_make_symbolic(&c, sizeof(c), "c");
    triangle(a, b, c);
    return 0;
}

void triangle(int a, int b, int c) {
    if ((a + b > c) && (a + c > b) && (b + c > a)) {
        if (a == b || a == c || b == c) {
            if (a == b && a == c) {
                printf("equilateral triangle.\n");
            } else {
                printf("isosceles triangle.\n");
            }
        } else {
            printf("triangle.\n");
        }
    } else {
        printf("non-triangle.\n");
    }
    return;
}' > program.c
```

This file was then compiled with the following command:
```
clang -I klee_src/include -emit-llvm -c -g -O0 -Xclang -disable-O0-optnone
program.c
```
This creates the output file `program.bc`

Klee was then run on the compiled file with the following command:
```
klee program.bc
```
This then created an output folder called `klee-out-0`

The created tests within that folder were then looked at with `ktest-tool` utility. An example of doing so on the first generated test is shown below:
```
ktest-tool test000001.ktest
```

Generated Test Inputs

KLEE generated 8 test cases. They are summarised in the table below, showing test number, input values, program output, and a short explanation of the execution path.

Test Number	**Input A**	**Input B**	**Input C**	**Expected Output**	**Path Coverage Explanation**
000001	0	0	0	non-triangle.	Fails first condition for not being a triangle.
000002	2	16777216	0	non-triangle.	Fails second condition for not being a triangle.
000003	16777216	0	0	non-triangle.	Fails third condition for not being a triangle.
000004	2048	128	2048	isosceles triangle.	Passes isosceles triangle test on second condition (a == c). Fails on the first condition to be an equilateral triangle. (a != b)
000005	16777216	16777216	16777216	equilateral triangle.	Equilateral triangle.
000006	1023410176	1023410176	33554432	isosceles triangle.	Passes isosceles triangle test on first condition (a == b). Fails on the second condition to be an equilateral triangle. (a != c)
000007	29360257	29360259	33554432	triangle.	Fails all conditions to be an isosceles triangle.
000008	14	2048	2048	isosceles triangle.	Passes isosceles triangle test on third condition (b == c). Fails on the first condition to be an equilateral triangle. (a != b)

Mutation testing

Mutation Operators Used

The mutation operators used in this to test this program were the following:

- Relational Operator Replacement (ROR)
- Arithmetic Operator Replacement (AOR)

Description of Generated Mutants

40 mutants were generated for ROR. These were generated by swapping the following relational operators for each other:

- ==
- !=
- <
- <=
- >
- >=

12 mutants were generated for AOR by swapping the following arithmetic operators for each other:

- +
- -
- /
- *
- %

Some examples of mutations are shown in the table below:

Original Condition	**Mutated Condition**	**Operator Used**
`a + b > c`	`a + b != c`	ROR
`a + b > c`	`a * b > c`	AOR
`a + b > c`	`a + b >= c`	ROR
`b + c > a`	`b % c > a`	AOR

Test Cases Used For Testing The Mutants

The test cases used for testing the mutants were the same test cases generated by symbolic execution. They are shown in the table below.

Test Number	Input A	Input B	Input C	Expected Output
000001	0	0	0	non-triangle.
000002	2	16777216	0	non-triangle.
000003	16777216	0	0	non-triangle.
000004	2048	128	2048	isosceles triangle.
000005	16777216	16777216	16777216	equilateral triangle.
000006	1023410176	1023410176	33554432	isosceles triangle.
000007	29360257	29360259	33554432	triangle.
000008	14	2048	2048	isosceles triangle.

Number of Mutants Killed

Mutation Operation	Mutants killed	Total mutants	Percentage of mutants killed
ROR	31	40	77.5%
AOR	12	12	100%
Total	43	52	≈ 82.7%

Mutation Score

The mutation score can be calculated using the formula: *mutation score = number of killed mutants / (total mutants - equivalent mutants)*. After manually reviewing the mutants there were no equivalent mutants. Using this we can then put in the results from the table above to get: *mutation score = 43 / (52 - 0)*. This gives us a mutation score of approximately 82.7%.

Use of AI Tools

Prompt 1:
Generate mermaid code that represents the modularity and flow of the following code: [Supplied Code Base]
Prompt 1 Response:
graph TD A[PacketFuzzer Main] -->|1. Requests Seed| B(SeedManager) A -->|2. Requests Mutation| C(Mutator) A -->|3. Dispatches Fuzz Input| D(Runner) D -->|4. Executes| E{PacketLab} E -->|5. Capture Output| D D -->|6. Send Results| F(CrashAnalyzer) F -.->|7. Save Unique Crash| G[Local Filesystem]
Prompt 1 Hallucinations: The generated response did not account for a few return points within the graph. I manually went through and followed the graph and code side by side, filling in the missing flow points and reordering when necessary.

Prompt 2:
Generate a graph depicting the function: y = max(10, x / 2)
Prompt 2 Response: Figure 2 Graph
Prompt 2 Hallucinations: This prompt produced no hallucinations as it was essentially just a tool call to matplotlib.

Prompt 3:
What's the best way to create a signature for hangs in this fuzzer?
Prompt 2 Response: If you want strongest dedup quality, capture the timed-out process thread dump (jcmd <pid> Thread.print) and use top app frames as the signature. That groups by actual stuck code path, not only input shape.